

Applied Cloud Computing

Assignment 4 : Part 1: Docker Desktop (updated Oct. 2025)

The main objective of this homework assignment is for you to learn how to set up a Container Platform type of virtualization environment via using Docker (<https://www.docker.com/>). While you follow the steps in this assignment, wherever you see the “**Answer**”:it means you need to fill in some data that being asked. You will be graded based on the information you fill in.

Step 1

General Reading:

<https://docs.docker.com/get-started/>

Install Docker Desktop on Mac:

<https://docs.docker.com/desktop/setup/install/mac-install/>

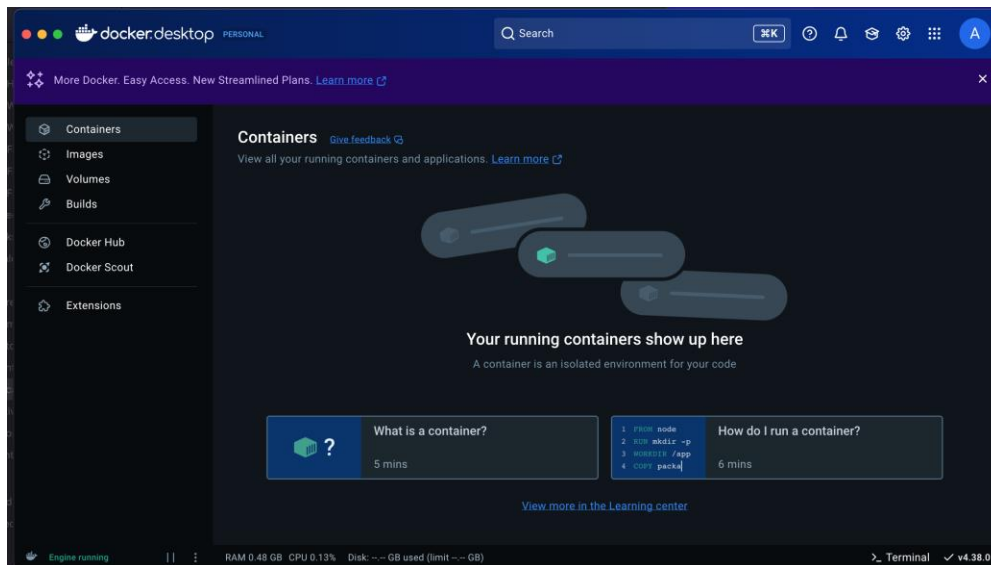
Install Docker Desktop on Windows:

<https://docs.docker.com/desktop/setup/install/windows-install/>

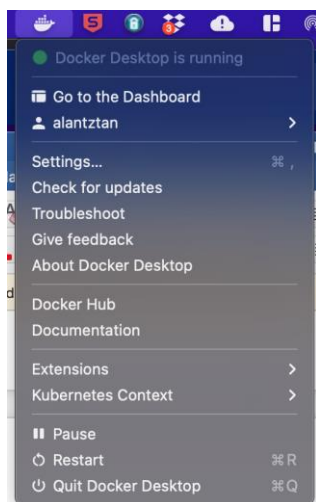
Install the Docker app based on instructions on screen.

Step 2

After a successful installation, launch the Docker app from your application folder. Click next. You need to allow ‘privileged access’ , agree ‘Docker Subscription Service Agreement’, then select “Personal use”, set up an account. ‘Docker desktop’ IDE will be opened. Also a ‘whale icon’ should appear in the top bar. You should see something like this:



If you click the ‘whale’ icon on top menu bar, you should see something like :



Step 3

Open a terminal and execute below commands:

- Check versions
 - **\$docker --version** (two "--")
 - **Answer:** Docker version 28.5.1, build e180ab8
 - **\$docker compose version**
 - **Answer:** Docker Compose version v2.40.0-desktop.1
 - **\$docker --help**
 - it will show you the on-line help

- **\$docker info**
 - it will display a lot of system-wide information. Within this info, from the “Server:” section, find out how
 - many Containers is there? Answer: 0
 - OSType: Answer: linux
 - Architecture: Answer: aarch64
 - CPUs: Answer: 8
 - Total Memory: Answer: 7.654GiB

Step 4

Let's start to execute some Docker run commands:

- to verify whether your installation is correct or not
 - **\$docker run hello-world**
 - If everything is correct, you should see something like below:


```
.....
Hello from Docker!
This message shows that your installation appears to be working correctly.

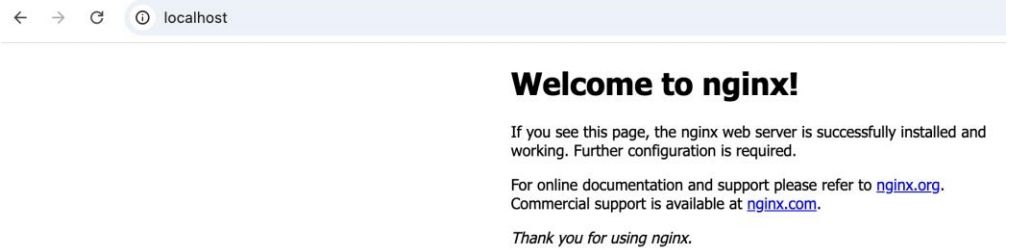
To generate this message, Docker took the following steps:
.....
```
 - Read carefully what steps had been performed for you.
 - How many Containers is there and what's their status?
 - Answer: 1 and Stopped
- Bring up an Ubuntu OS's bash shell:
 - **\$docker run -it ubuntu bash**
 - What the option -i and -t stands for?
 - Answer: The -i option keeps the container's input open so you can type commands, and the -t option gives you a terminal-like screen to interact with. Together, -it lets you open an interactive shell inside the container, so you can use it just like a normal Linux terminal.
 - You should see your terminal window's prompt got changed to something like "root@bce48bd17394:/#" . This means you are in an

Ubuntu OS's bash shell (and NOT in your host OS terminal/command prompt anymore)

- You can execute
 - **"uname --a"**
 - it will print lots of information regarding the Ubuntu OS. Copy the information you see into below:
 - **Answer:** Linux 6b023741ab4e 6.10.14-linuxkit #1 SMP Wed Sep 10 06:47:45 UTC 2025 aarch64 aarch64 aarch64 GNU/Linux
 - **"cd /bin"**
 - it will change directory to /bin
 - **"ls -als"**
 - it will show you the content of that directory
 - **"pwd"**
 - it will show you the directory where you are now
 - In another terminal window, use 'docker info' cmd and find out what's the current container status
 - **Answer:** 2 containers, 1 running, 1 stopped
 - **"exit"**
 - will terminate the bash shell and return you back to the original terminal/cmd prompt.
- In your own words, explain what had happened when you execute the **"\$docker run -it ubuntu bash"** ?
 - **Answer:** When you run \$ docker run -it ubuntu bash, Docker first checks if the Ubuntu image is available on your system; if not, it downloads it. Then, it creates a new container from that image and starts it. The -it part makes sure you get an interactive terminal connected to the container. Finally, it runs the bash command inside the container, which opens a Linux shell where you can type and run commands just like you would on a regular computer.

- Start a dockerized web server
 - **\$docker run -d -p 80:80 --name webserver nginx**
 - The above command will start a 'dockerized' webserver on your local machine port 80.

- To verify it, in a web browser issue: <http://localhost/>
 - Screen capture what you see from the web browser to below:
 - Answer:



Step 5

Now let's check how many Docker containers we already brought up with 'container' option:

- List Docker containers which are running

- **\$docker container ls**

```
((base) shwetaleegadekar@Shwetalees-MacBook-Air-3143 ~ % docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
5f2eb52e3d6b   nginx    "/docker-entrypoint...." About a minute ago Up About a minute   0.0.0.0:80->80/tcp, [::]:80->80/tcp  webserver
```

- Execute this command:

- **\$docker container ls -aq**

- What is the option -a and -q stands for?
- **Answer:** The command docker container ls -aq lists all containers on your system, both running and stopped, because of -a, and shows only their IDs, without any extra details, because of -q.

- List all Docker containers

- **\$docker container ls -a**

- pay attention to "CONTAINER ID" (the first column) and the "NAMES" (the last column)
- "CONTAINER ID" is a hash ID
 - What is a hash ID?
 - **Answer:** A hash ID in Docker is a long, unique string of letters and numbers that Docker automatically generates to identify each container. It's created using a hashing algorithm, which makes it almost impossible for two containers to have the same ID. Think of it like a fingerprint for the container — it's how Docker keeps track of each one, even if multiple containers are running from the same image.
- When you create a new Docker container and don't give a custom name (by passing --name <name> on the CLI) Docker generates a name for you.
- Now take a closer look at the 'NAMES' column. Do your own self-study and provide what you find out why the 'NAMES' column looks that way. You can include 1 or 2 of the URLs you referenced.
 - **Answer:** When you don't give a container a name yourself, Docker automatically creates one for you. It does this by combining two words: usually a random adjective and the last name of a famous scientist, hacker, or inventor (for example, sleepy_stonebraker or great_murdock). This makes the names human-readable and unique, so you don't have to rely only on the long hash IDs to identify your containers.
If you want more control, you can set a name when creating a container using the --name option (like --name webserver). This

way, instead of random names, you can give containers meaningful labels that are easier to remember and use in commands

- For all commands you later run that interact with the container you can either use its hash ID or its name.

- Stop and remove containers with the following commands:

- ***\$docker container stop webserver***

- This will stop the container with NAME="webserver"
- If you issue ***\$docker container ls*** again, then you should see the webserver container is no longer running.

- ***\$docker container ls -a***

- You should still see the IMAGE 'ubuntu', 'nginx' and 'hello-world' containers are still there but STATUS is in 'Exited' state.
- Take a screenshot shows 3 images' information:

- **Answer:**

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3143 ~ % docker container ls -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS              PORTS          NAMES
5f2eb52e3d6b   nginx      "/docker-entrypoint...." 23 minutes ago Exited (0) About a minute ago
6b023741ab4e   ubuntu    "bash"                  29 minutes ago Exited (0) 24 minutes ago
20ff0a67fa6c   hello-world "/hello"                36 minutes ago Exited (0) 36 minutes ago
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3143 ~ %
```

- ***\$docker container rm webserver***

- This will actually remove the container NAME 'webserver'

- ***\$docker container prune***

- This will remove all stopped containers

- Most of the above operations you can also perform in the docker desktop in UI mode. Pls redo previous steps in UI mode.

- Manage container images:

- ***\$docker image ls***

- This will list all local container images (now you should have at least 3, nginx, hello-world and ubuntu)
- Take a screenshot shows 3 images' information:

- **Answer:**

```
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3143 ~ % docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest   3b7732505933   13 days ago   255MB
ubuntu        latest   66460d557b25   2 weeks ago   139MB
hello-world    latest   6dc565aa6309   2 months ago  16.9kB
(base) shwetaleegadekar@Shwetalees-MacBook-Air-3143 ~ %
```

- **`$docker image rm nginx`**
 - This will remove image with REPOSITORY name 'nginx'

Step 6

Let's manually install the Container CentOS:

- **`$docker pull centos:8`**
 - This will pull the older version of centos from online repository into local repository (`centos:latest` image no longer exists on Docker Hub, so we have to pull an older version)
 - Use cmd **`$docker image ls`** to check whether it is in local repository
 - How big is the centos image?
 - **Answer:** 380MB
- Bring up a Centos bash shell **`$docker run -it centos:8 bash`**
 - If you didn't pull the centos image into your local repository and directly execute the **`$docker run -it centos:8 bash`**, what will happen ?
 - **Answer:** Without having the centos:8 image already on your computer, Docker will first **check your local image repository**. Since it won't find centos:8 there, it will automatically **pull (download) the image from Docker Hub** (the default public image registry). After the image finishes downloading, Docker will then **create a new container** from that image and open an **interactive bash shell** inside it.
- In this Centos Bash command prompt, execute **`#top`**
 - **top** command displays processor activity of your Linux box and also displays tasks managed by kernel in real-time.
 - Look at all fields then exit by pressing **Ctrl-C**
- In bash command issue **`#yes > /dev/null`** then press **Ctrl-Z** to suspend the task, then issue **`#bg %`** . Or just execute **`#yes > /dev/null &`**
This sequence of commands will bring the 'yes > /dev/null' to background and execute in full speed.
- Now issue the top command again in the same Centos command prompt.
 - How much %CPU command 'yes' is occupying?
 - **Answer:** 100
- Bring up the docker desktop IDE, click the 'Containers' menu on the left side, observe how much CPU CentOS is using. Perform a screen capture and put in below 'Answer'. (If your system has multi-core, you should record something like X% / Y% (Z CPUs available))
 - Docker Desktop screenshot:
Answer:

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last st	Actions
☐	compassionate_	aeab01312c07	centos:8		0%	34 min	
☐	zen_wiles	29b117aa1cc6	centos/cen		0%	21 min	
☐	beautiful_pasca	617e4a0a7f02	centos/cen		0%	20 min	

- How busy is the %CPU? (If your system has multi-core, you should record something like X% / Y% (Z CPUs available)
Answer: 0% / 800%
- How much %CPU command 'yes' is occupying?
Answer: 100
- You can Issue **#jobs** command to see how many tasks are running in centos.
- Exit the top command and issue the 2nd **#yes > /dev/null** command then bring it to the background and let it run full speed (or directly issue **#yes > /dev/null &**) Observe the CPU usage by using top command again.
 - How many 'yes' command is running?
 - Answer: 2
 - How much %CPU each 'yes' command is occupying?
 - Answer: 100
 - How busy is the overall %CPU from docker desktop?
 - Answer: The overall CPU usage is about 200% / 800% (8 CPUs available), which means the two yes processes are each fully occupying one CPU core.
- Exit the top command and issue the 3rd **#yes > /dev/null &** command, bring it to the background and let it run full speed. Observe the CPU usage by using top command again.
 - How many 'yes' command is running?
 - Answer: 3
 - How much %CPU each 'yes' command is occupying?
- Answer: One yes process is using **100.0% CPU**
 Another yes process is using **100.0% CPU**
 A third yes process is using **99.7% CPU**

- How busy is the host computer's %CPU?
 - Answer: The host computer's CPU is about **300% used in total**, which equals ~37% of all CPU capacity on an 8-core system (or ~75% on a 4-core system).
- Exit the top command and issue the 4th **`#yes > /dev/null`** command then bring it to the background and let it run full speed. Observe the CPU usage by using top command again.
 - How many 'yes' command is running?
 - Answer: 4
 - How much %CPU each 'yes' command is occupying?
 - Answer:
- One yes process is using **100.0% CPU**
 Another yes process is using **100.0% CPU**
 A third yes process is using **99.7% CPU**
 A fourth yes process is using **99.7% CPU**
 - How busy is the host computer's %CPU?
 - Answer: The host computer's CPU is about 400% used in total, which equals roughly 50% busy on an 8-core system.
- Repeat the above procedure until you created 1 more task than the number of your local multi-core system supports. For example, in my MacBook, I have 10 CPUs, so I created 11 tasks (**`yes > /dev/null & put it into background`**).
 - How many processors in your system?
 - Answer:8
 - Bring up 'top' cmd in the terminal, perform screen capture the top cmd screen. Paste the screenshot below:
 - Answer:

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
30	root	20	0	4996	2016	1888	R	96.3	0.0	1:13.95	yes
24	root	20	0	4996	1984	1856	R	92.7	0.0	5:03.66	yes
22	root	20	0	4996	2128	2000	R	91.7	0.0	7:12.99	yes
20	root	20	0	4996	2204	2076	R	90.4	0.0	9:26.77	yes
26	root	20	0	4996	1984	1856	R	88.0	0.0	1:19.76	yes
28	root	20	0	4996	2204	2076	R	85.4	0.0	1:17.69	yes
29	root	20	0	4996	2144	2016	R	85.4	0.0	1:16.68	yes
17	root	20	0	4996	1992	1864	R	84.4	0.0	11:10.34	yes
27	root	20	0	4996	2204	2076	R	82.4	0.0	1:17.97	yes
1	root	20	0	3788	2676	2420	S	0.0	0.0	0:00.16	bash
16	root	20	0	4996	2016	1888	T	0.0	0.0	0:10.17	yes
32	root	20	0	9116	3208	2696	R	0.0	0.0	0:00.00	top

- Based on the above observations, what kind of conclusion you can give:
 - Answer:** Running several yes processes in the background quickly drives CPU usage close to 100% per process, effectively maxing out multiple CPU cores. This demonstrates how lightweight commands can still saturate CPU resources and why process management (foreground/background jobs) is important in Linux.
- Now you can exit the Centos command prompt by executing 'exit'

Now we want to reduce the CPU usage for the CentOS container from default(max) to a lower number, for example change from 10 to 4. Use the following cmd to restart the CentOS container:

#docker run -it --cpus="4" centos:8 /bin/bash

After restarting CentOS container, redo the steps in Step 6. Take a last top cmd screenshot, write down the differences:

- Answer:**

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
23	root	20	0	4996	2128	2000	R	50.2	0.0	1:13.26	yes
24	root	20	0	4996	1992	1864	R	50.2	0.0	1:12.98	yes
19	root	20	0	4996	2204	2076	R	49.8	0.0	1:19.39	yes
21	root	20	0	4996	2204	2076	R	49.8	0.0	1:14.79	yes
22	root	20	0	4996	1984	1856	R	49.8	0.0	1:14.07	yes
25	root	20	0	4996	2128	2000	R	49.2	0.0	1:12.36	yes
26	root	20	0	4996	2120	1992	R	49.2	0.0	1:11.79	yes
20	root	20	0	4996	2048	1920	R	25.2	0.0	0:40.05	yes
27	root	20	0	4996	2120	1992	R	24.9	0.0	0:35.67	yes
1	root	20	0	3788	2796	2412	S	0.0	0.0	0:00.06	bash
28	root	20	0	9116	3316	2804	R	0.0	0.0	0:00.03	top

- **Answer:** Even though the number of yes processes stayed the same (9), in the earlier case they were each hogging nearly 100% CPU, while now the OS/Docker scheduler is splitting CPU time among them, so each process only shows ~50% CPU.

After above exercises are done, don't forget to remove all running containers from your local machine! Make sure follow cmd return no container.

- ***\$docker container ls***

Also verify in the docker desktop in UI mode.

This concludes this Part 1 exercise. There will be Part 2 so keep the docker app and all the Images already downloaded.